P_151:Credit Card Fraud Rule Engine — Project Documentation

---

1. Purpose

The Credit Card Fraud Rule Engine is designed to detect, flag, and prevent fraudulent credit card transactions in real time by evaluating each transaction against a configurable set of business rules. The system aims to:

- Reduce financial losses due to fraudulent transactions.
- Provide a transparent, auditable, and explainable decision process (as opposed to a pure black-box ML model).
- Allow fraud analysts to create, modify, enable/disable, and prioritize rules without requiring code changes or redeployment.
- Support real-time transaction scoring and near-instant approve/decline/review decisions.
- Maintain a complete audit trail of transactions, rule evaluations, and analyst actions for compliance and reporting purposes.
- Provide dashboards and reports for monitoring fraud trends, rule performance, and false-positive rates.

2. Technology Stack

Layer	Technology
Backend Framework	Python, FastAPI
Frontend	Python, Streamlit
Database	MySQL / PostgreSQL
ORM	SQLAlchemy (with Alembic for migrations)
Data Validation / Schemas	Pydantic
ASGI Server	Uvicorn (optionally behind Gunicorn for production)
Dependency/Environment Management	pip + venv, or Poetry
Authentication & Authorization	FastAPI Security utilities + JWT (via `python-jose`/`PyJWT`), OAuth2PasswordBearer
API Style	RESTful APIs (JSON over HTTPS), auto-generated OpenAPI/Swagger docs
Rule Engine	Custom Python rule evaluator (or a library such as `durable_rules`/`business-rules`)
Messaging/Streaming (optional, for real-time ingestion)	Apache Kafka (via `aiokafka` or `confluent-kafka-python`)
Version Control	Git / GitHub
Testing	Pytest, `httpx`/`TestClient` for API testing
Deployment	Docker containers, hosted on a cloud platform (AWS/Azure/GCP) or on-premise server
Logging & Monitoring	Python `logging` module, optionally ELK stack

3. Database Schema

Core tables:

users

- user_id (PK)
- username
- password_hash
- email
- role (ADMIN / ANALYST / VIEWER)
- created_at

customers

- customer_id (PK)
- name
- email
- phone
- registered_address
- account_created_at

cards

- card_id (PK)
- customer_id (FK → customers)
- card_number_masked
- card_type
- issue_date
- expiry_date
- status (ACTIVE / BLOCKED / EXPIRED)

transactions

- transaction_id (PK)
- card_id (FK → cards)
- amount
- currency
- merchant_id (FK → merchants)
- transaction_time
- location
- device_fingerprint
- ip_address
- status (APPROVED / DECLINED / UNDER_REVIEW)
- risk_score

merchants

- merchant_id (PK)

- merchant_name
- mcc_code (merchant category code)
- merchant_location
- risk_category

rules

- rule_id (PK)
- rule_name
- rule_description
- rule_type (VELOCITY / GEOLOCATION / AMOUNT_THRESHOLD / MCC / DEVICE / BEHAVIORAL / BLACKLIST)
- rule_expression (condition logic, stored as JSON/text)
- weight/score
- is_active (boolean)
- created_by (FK → users)
- created_at
- updated_at

rule_evaluations

- evaluation_id (PK)
- transaction_id (FK → transactions)
- rule_id (FK → rules)
- triggered (boolean)
- score_contribution
- evaluated_at

fraud_cases

- case_id (PK)
- transaction_id (FK → transactions)
- assigned_analyst_id (FK → users)
- case_status (OPEN / IN_REVIEW / CLOSED_FRAUD / CLOSED_LEGITIMATE)
- notes
- created_at
- resolved_at

audit_logs

- log_id (PK)
- user_id (FK → users)
- action_performed
- entity_affected
- timestamp

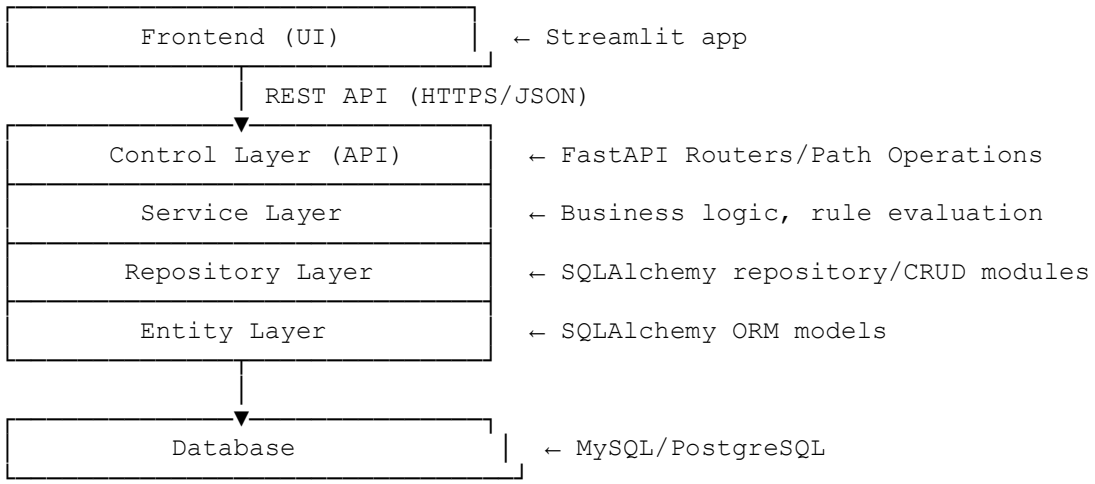
4. Database

- **Type:** Relational (RDBMS) — MySQL or PostgreSQL, chosen for strong ACID compliance (critical for financial data integrity).

- **Normalization:** Schema normalized to 3NF to reduce redundancy while keeping join complexity manageable for real-time lookups.
- **Indexing:** Indexes on `card_id`, `transaction_time`, `merchant_id`, and `customer_id` to speed up velocity and pattern-based rule checks.
- **Partitioning:** Transaction table can be partitioned by date range for scalability as volume grows.
- **Backup Strategy:** Scheduled automated backups with point-in-time recovery for regulatory compliance.

5. System Architecture

The system follows a **layered (N-tier) architecture**:



Transaction data flows in through an API/ingestion endpoint, is evaluated by the rule engine in the service layer, persisted, and — if flagged — routed into the case management workflow for analyst review.

6. Backend Module (under System Architecture)

The backend is organized into modules:

- **Auth Module** — login, JWT issuance, role-based access control.
- **Transaction Module** — ingestion, validation, and persistence of transactions.
- **Rule Engine Module** — loads active rules, evaluates each transaction against them, computes a composite risk score.
- **Case Management Module** — creates and tracks fraud cases for flagged transactions.
- **Reporting/Analytics Module** — aggregates data for dashboards (fraud trend charts, rule trigger frequency, false-positive rate).
- **Audit Module** — logs every rule change, analyst decision, and system action.

6.1 Entity Layer

SQLAlchemy ORM model classes (declarative `Base` subclasses) that map directly to database tables — e.g., `Transaction`, `Card`, `Rule`, `Customer`, `FraudCase`, defined with `Column`, `ForeignKey`, and

`relationship()`. Pydantic schema classes (`BaseModel`) sit alongside these as request/response DTOs for the API layer. This layer defines the data model and relationships only; it contains no business logic.

6.2 Repository Layer

Python modules/classes (e.g., `TransactionRepository`, `RuleRepository`) that wrap a SQLAlchemy `Session` and provide CRUD functions and custom queries (e.g., `get_transactions_by_card_in_range(card_id, start, end)`), used by the service layer to fetch and persist data without embedding raw SQL or ORM calls throughout the business logic.

6.3 Service Layer

Contains the core business logic, implemented as plain Python service classes/functions:

- `RuleEngineService` — evaluates a transaction against all active rules and computes a risk score.
- `TransactionService` — orchestrates transaction intake, calls the rule engine, and updates transaction status.
- `FraudCaseService` — manages the lifecycle of flagged cases.
- `UserService` — handles authentication/authorization logic.

This layer is invoked via FastAPI's dependency injection (`Depends`) and wraps each operation in a database transaction (commit/rollback) to ensure business rules are enforced consistently regardless of which route invokes it.

6.4 Control Layer

FastAPI routers (`APIRouter`) that expose API endpoints, e.g.:

- `POST /api/transactions` — submit a new transaction for evaluation.
- `GET /api/rules` — list all rules.
- `POST /api/rules` — create a new rule.
- `PUT /api/rules/{id}` — update/toggle a rule.
- `GET /api/cases` — list fraud cases for review.
- `POST /api/auth/login` — authenticate a user.

Request/response bodies are validated automatically via Pydantic models, and interactive API documentation (Swagger UI / ReDoc) is generated automatically by FastAPI. The control layer handles HTTP request/response mapping and input validation, and delegates all business logic to the service layer via dependency injection.

7. Frontend Pages (Streamlit)

- **Login Page** — user authentication (Streamlit session state holds the JWT after calling the auth API).
- **Dashboard** — summary widgets (transactions today, fraud flagged, rule trigger stats, risk score trends) via `st.metric` and charts.
- **Transactions Page** — searchable/filterable table of transactions with status and risk score, built with `st.dataframe`/`st.data_editor`.
- **Transaction Detail Page** — full detail of a transaction plus which rules triggered and why, shown via expandable sections.
- **Rule Management Page** — create, edit, enable/disable, and prioritize rules through Streamlit forms (`st.form`); view rule performance metrics.

- **Fraud Case Management Page** — list of open/closed cases assigned to analysts, with notes and resolution actions.
- **Reports/Analytics Page** — charts for fraud trends, false-positive rate, rule effectiveness over time, using `st.line_chart`/`st.bar_chart` or Plotly/Matplotlib embedded in Streamlit.
- **User/Role Management Page** (admin only) — manage analyst and admin accounts.
- **Audit Log Page** — view of all system and user actions for compliance.

Each page communicates with the FastAPI backend purely over HTTP (via `requests`/`httpx`), keeping the frontend a thin, all-Python client with no JavaScript, HTML, or CSS involved.

8. Relationships (Entity Relationship Summary)

- A **Customer** has many **Cards** (1:N).
- A **Card** has many **Transactions** (1:N).
- A **Merchant** has many **Transactions** (1:N).
- A **Transaction** is evaluated against many **Rules**, tracked via **rule_evaluations** (M:N, resolved through a join table).
- A **Transaction** may result in zero or one **Fraud Case** (1:0..1).
- A **Fraud Case** is assigned to one **User/Analyst**, and a User can handle many Cases (1:N).
- A **User** performs many **Audit Log** entries (1:N).
- A **Rule** is created/modified by a **User** (N:1) and generates many **rule_evaluations** (1:N).

9. Data Protection

- **Encryption at rest** — sensitive fields (card numbers) stored masked/tokenized, never in plaintext; database-level encryption (TDE) where supported.
- **Encryption in transit** — all API traffic over HTTPS/TLS.
- **PCI-DSS alignment** — following Payment Card Industry Data Security Standard practices: no storage of full PAN or CVV, restricted access to cardholder data, regular security testing.
- **Authentication & Authorization** — JWT-based stateless auth; role-based access control (Admin, Analyst, Viewer) restricting who can view/modify rules or cases.
- **Password Security** — hashed with a strong algorithm (BCrypt).
- **Audit Trails** — every sensitive action (rule change, case resolution, login) is logged immutably.
- **Data Masking** — UI displays masked card numbers (e.g., `**** **** **** 1234`).
- **Rate Limiting / API Gateway** — protects ingestion endpoints from abuse.
- **Regular Backups & Disaster Recovery** — encrypted backups with defined RTO/RPO.

10. Connection

- **Frontend ↔ Backend:** Streamlit app calls the FastAPI REST API over HTTPS using `requests`/`httpx`, attaching the JWT bearer token (kept in Streamlit `session_state`) on each request.
- **Backend ↔ Database:** Connection pooling via SQLAlchemy's built-in connection pool; credentials stored securely (environment variables via `python-dotenv`/`pydantic-settings`, or a secrets manager — not hardcoded).
- **Backend ↔ Rule Engine:** Internal service call (in-process) or, if externalized, via a lightweight internal API/gRPC call to a dedicated rule-evaluation microservice.
- **Real-time ingestion (optional):** Transaction events published to a Kafka topic; the rule engine service consumes the stream for near-real-time evaluation at scale.

- **External Integrations:** Optional connectors to payment gateways/card networks for live transaction feeds, and to notification services (email/SMS) for alerting customers or analysts.

11. Novelty / What's New in This Project

While built on established fraud-detection concepts, this project differentiates itself in several ways:

1. **Explainable, analyst-controlled rules (not a black box)** — Unlike pure ML fraud systems, every decision traces back to the specific rules that triggered it, with individual score contributions logged in `rule_evaluations`. Analysts and auditors can see exactly *why* a transaction was flagged.
2. **Rules configurable at runtime, no redeployment needed** — Rules live in the database (`rules` table) with an `is_active` flag and a stored `rule_expression`, so fraud analysts can create, tune, or disable a rule through the UI without a developer touching code or redeploying the application.
3. **Composite risk scoring instead of single-rule blocking** — Rather than one rule causing an outright block, each triggered rule contributes a weighted score; the transaction is approved, sent for step-up authentication, or declined based on the *cumulative* score, reducing false positives from any single overly strict rule.
4. **Full case-management workflow built in** — Flagged transactions don't just get logged; they flow into a `fraud_cases` lifecycle (OPEN → IN_REVIEW → CLOSED_FRAUD/CLOSED_LEGITIMATE) assigned to analysts, which many basic/demo projects skip entirely.
5. **Compliance-first data handling** — Card numbers are masked/tokenized at rest, PCI-DSS-aligned practices are followed, and an immutable `audit_logs` table captures every rule change and analyst action, treating the system as production/regulatory-grade rather than a toy demo..

12. Future Scope and Enhancements

- Integrate a **machine learning risk-scoring model** alongside the rule engine for a hybrid rules + ML approach, catching subtler fraud patterns rules alone might miss.
- Add a **rule simulation/backtesting sandbox** so analysts can test new rules against historical transaction data before activating them in production.
- Implement **adaptive/self-tuning rule weights** based on feedback from confirmed fraud vs. false positives.
- Build a **mobile app** for customers to receive instant fraud alerts and confirm/deny flagged transactions.
- Add **graph-based analysis** to detect fraud rings (linked accounts, shared devices/IPs across multiple cardholders).
- Support **multi-currency and cross-border transaction rules** with region-specific compliance logic.